

RAPPORT TECHNIQUE

Puls-Events

Assistant Intelligent pour la Recommandation d'Événements Culturels

Système RAG — Retrieval-Augmented Generation

Auteur	Haroun
Client	Puls-Events
Version	v1.0.0
Date	Juin 2026
Statut	POC — Proof of Concept

Sommaire

1. Contexte et objectifs
2. Architecture du système
3. Données utilisées
4. Choix technologiques
5. Modèles utilisés
6. Pipeline RAG
7. API REST
8. Tests et couverture
9. Évaluation et résultats
10. Déploiement Docker
11. Pistes d'amélioration
12. Conclusion

1. Contexte et objectifs

Puls-Events est une plateforme technologique spécialisée dans les recommandations culturelles personnalisées. Face à la volumétrie croissante des événements culturels disponibles, les utilisateurs peinent à identifier rapidement ceux qui correspondent à leurs centres d'intérêt.

Ce POC (Proof of Concept) a pour objectif de démontrer la faisabilité technique d'un chatbot intelligent capable de répondre à des questions en langage naturel sur les événements culturels parisiens, en s'appuyant sur un système RAG combinant recherche vectorielle et génération de réponse par LLM.

Objectifs techniques

- Récupérer et structurer les événements culturels parisiens via l'API Open Agenda
- Indexer ces événements sous forme de vecteurs sémantiques dans une base FAISS
- Générer des réponses pertinentes et contextualisées via le modèle Mistral Large
- Exposer le système via une API REST documentée (FastAPI)
- Conteneuriser la solution avec Docker pour un déploiement reproductible
- Valider la qualité du système avec des tests unitaires et un jeu de test annoté

2. Architecture du système

Le système repose sur une architecture RAG (Retrieval-Augmented Generation) en trois couches principales : la collecte et l'indexation des données, le moteur de recherche sémantique, et la génération de réponses en langage naturel.

Vue d'ensemble

Couche	Composant	Technologie	Rôle
Données	Collecte	Open Agenda API	Récupération des événements
Données	Stockage	Pandas + CSV	Structuration et nettoyage
Vectorisation	Embeddings	Mistral Embed	Conversion texte → vecteurs
Vectorisation	Index	FAISS IndexFlatL2	Recherche par similarité
Génération	LLM	Mistral Large Latest	Génération des réponses
API	Serveur	FastAPI + Uvicorn	Exposition du système
Déploiement	Conteneur	Docker	Packaging reproductible

Flux de traitement

1. L'utilisateur envoie une question via HTTP POST sur le endpoint /ask
2. La question est vectorisée via l'API Mistral Embed (dimension 1024)
3. FAISS recherche les 5 événements les plus proches sémantiquement
4. Les événements trouvés sont injectés dans un prompt structuré
5. Mistral Large génère une réponse en langage naturel
6. La réponse est retournée avec les sources citées en JSON

3. Données utilisées

Les données proviennent de l'API publique Open Agenda, plateforme de référence pour les événements culturels en France. La collecte a ciblé la région parisienne sur une fenêtre d'un an.

Statistiques du jeu de données

Indicateur	Valeur
Source	API Open Agenda
Zone géographique	Paris et Île-de-France
Nombre d'agendas	10 agendas parisiens
Événements collectés	777 événements
Format de stockage	CSV (data/events.csv)
Période couverte	Juin 2025 — Juin 2026

Agendas ciblés

- Diocèse de Paris — événements religieux et concerts sacrés
- Maison de l'Europe de Paris — conférences et débats européens
- Université Paris-Saclay — séminaires académiques
- JASS CLUB PARIS — événements jazz
- Faculté de Médecine Paris-Saclay — conférences médicales
- Grand Paris Seine Ouest — événements culturels locaux
- Conservatoires de Grand Paris Seine Ouest — concerts et ateliers
- Grand Paris Grand Est — événements européens
- Librairie de Paris Place de Clichy — rencontres littéraires
- FICEP — événements culturels étrangers

Structure des données

Champ	Type	Description
id	Integer	Identifiant unique Open Agenda
titre	String	Titre de l'événement
description	String	Description complète (texte vectorisé)

lieu	String	Nom du lieu
ville	String	Ville
adresse	String	Adresse complète
date_debut	String	Date et heure de début
date_fin	String	Date et heure de fin
url	String	URL de l'événement sur Open Agenda

4. Choix technologiques

Technologie	Version	Justification
Python	3.11	Stabilité, compatibilité, écosystème ML riche
Mistral AI	1.2.5	LLM et embeddings performants, API simple
FAISS	1.7.4	Recherche vectorielle rapide, portable (CPU)
FastAPI	0.111	Performance, documentation Swagger automatique
Pandas	2.0	Manipulation efficace des données tabulaires
Docker	latest	Conteneurisation pour déploiement reproductible
pytest	8.0	Framework de tests robuste et extensible
python-dotenv	1.0	Gestion sécurisée des variables d'environnement

Pourquoi FAISS plutôt qu'une base vectorielle cloud ?

FAISS (Facebook AI Similarity Search) a été choisi pour sa légèreté, sa portabilité et ses performances en local. Dans le cadre d'un POC, il permet de démarrer sans infrastructure cloud, tout en restant facilement remplaçable par Pinecone, Weaviate ou Qdrant en production.

Pourquoi Mistral plutôt qu'OpenAI ?

Mistral AI offre des modèles compétitifs avec une API accessible et des crédits gratuits suffisants pour un POC. Le modèle mistral-embed produit des embeddings de dimension 1024, bien adaptés à la recherche sémantique sur des descriptions d'événements en français.

5. Modèles utilisés

mistral-embed — Modèle d'embeddings

Paramètre	Valeur
Nom du modèle	mistral-embed
Dimension	1024
Langue	Multilingue (français optimisé)
Usage	Vectorisation des événements et des questions
Similarité	Distance L2 (FAISS IndexFlatL2)

mistral-large-latest — Modèle de génération

Paramètre	Valeur
Nom du modèle	mistral-large-latest
Type	LLM génératif
Usage	Génération des réponses en langage naturel
Contexte	Prompt avec événements + question utilisateur
Langue	Français

Stratégie de prompt

Le prompt injecté dans Mistral Large est structuré en trois parties : un rôle système définissant l'assistant comme expert en événements culturels parisiens, un contexte contenant les 5 événements les plus pertinents trouvés par FAISS, et la question de l'utilisateur. Cette approche garantit des réponses ancrées dans les données réelles.

6. Pipeline RAG

Phase 1 — Indexation (offline)

Le script `scripts/build_index.py` réalise les opérations suivantes :

- Chargement des 777 événements depuis `data/events.csv`
- Construction d'un chunk par événement : titre + description + lieu + ville + date
- Vectorisation par batch de 20 via l'API Mistral Embed
- Création de l'index FAISS `IndexFlatL2` (distance euclidienne)
- Sauvegarde de l'index, des chunks et des métadonnées dans `vectorstore/faiss_index/`

Phase 2 — Interrogation (online)

La classe `RAGSystem` dans `scripts/rag.py` gère le flux temps réel :

- Réception de la question utilisateur
- Vectorisation de la question (1 appel Mistral Embed)
- Recherche des 5 voisins les plus proches dans FAISS (Top-K=5)
- Construction du prompt avec les 5 événements comme contexte
- Génération de la réponse via Mistral Large (1 appel)
- Retour de la réponse + sources en JSON

Format d'un chunk

```
Titre: Concert COGE Description: Concert de musique sacrée... Lieu: Eglise  
Saint-Marcel Ville: Paris Date: 2026-06-12T20:00:00
```

7. API REST

L'API est développée avec FastAPI et exposée via Uvicorn. La documentation Swagger est automatiquement générée à l'adresse <http://localhost:8000/docs>.

Endpoint	Méthode	Description	Paramètres
/	GET	Vérification API	—
/health	GET	Statut + nb événements	—
/ask	POST	Question → Réponse RAG	question (string)
/rebuild	POST	Reconstruction index FAISS	—

Exemple d'appel /ask

```
curl -X POST "http://localhost:8000/ask" \ -H "Content-Type: application/json" \ -d '{"question": "Quels concerts sont prevus a Paris ?"}'
```

Exemple de réponse

```
{ "question": "Quels concerts sont prevus a Paris ?", "answer": "Voici les concerts prevus : Concert COGE...", "sources": [ {"titre": "Concert COGE", "lieu": "Eglise Saint-Marcel", "ville": "Paris", "date_debut": "2026-06-12"} ] }
```

8. Tests et couverture

Les tests unitaires couvrent l'intégralité du code avec un taux de couverture de 100%, validé par pytest-cov.

Fichier	Statements	Couverture	Tests couverts
api/main.py	50	100%	Endpoints, erreurs, rebuild
scripts/build_index.py	44	100%	Chunking, embeddings, FAISS
scripts/fetch_events.py	64	100%	Collecte, parsing, pipeline
scripts/rag.py	51	100%	Recherche, génération, demo
TOTAL	209	100%	35 tests

Stratégie de test

- Tous les appels API externes (Mistral, Open Agenda) sont mockés pour des tests reproductibles sans connexion internet
- Les fixtures pytest autouse garantissent l'isolation de chaque test
- Les blocs if `__name__ == '__main__'` sont marqués `# pragma: no cover`
- Les tests couvrent les cas nominaux, les cas limites et les erreurs

Lancer les tests

```
pytest tests/test_rag.py -v --cov=scripts --cov=api --cov-report=term-missing
```

9. Évaluation et résultats

Un jeu de 10 questions/réponses annotées manuellement (tests/test_set.json) permet d'évaluer automatiquement la qualité des réponses générées via un score de similarité cosinus sur les embeddings Mistral.

Résultats détaillés

#	Question	Score	Évaluation
01	Quels concerts sont prévus à Paris ?	0.9413	Correcte
02	Y a-t-il des événements pour les enfants ?	0.9014	Correcte
03	Quels événements culturels en juin 2026 ?	0.9220	Correcte
04	Événements à l'Église Saint-Marcel ?	0.9227	Correcte
05	Y a-t-il des événements jazz à Paris ?	0.9037	Correcte
06	Activités de la Maison de l'Europe ?	0.9211	Correcte
07	Événements littéraires à Paris ?	0.8355	Partielle
08	Événements religieux à Paris ?	0.8994	Partielle
09	Événements à l'université Paris-Saclay ?	0.8927	Partielle
10	Événements culturels étrangers à Paris ?	0.8555	Partielle

Synthèse des résultats

Métrique	Valeur
Score moyen de similarité	0.8995
Réponses correctes (≥0.90)	6 / 10 (60%)
Réponses partielles (≥0.75)	4 / 10 (40%)
Réponses incorrectes (<0.75)	0 / 10 (0%)
Événements indexés	777
Couverture tests unitaires	100% (35 tests)

Analyse

Le score moyen de 0.8995 est très proche du seuil de correction (0.90), ce qui valide la pertinence du système RAG pour ce POC. L'absence de réponse incorrecte est un indicateur fort de la fiabilité du système. Les 4 réponses partiellement correctes concernent des thématiques moins représentées dans le corpus (littérature, culture étrangère, religion), ce qui suggère un enrichissement des données comme principal axe d'amélioration.

10. Déploiement Docker

Le système est entièrement conteneurisé via Docker, garantissant la reproductibilité du déploiement sur n'importe quelle machine.

Build et lancement

```
# Démarrer Colima (Mac) colima start --dns 8.8.8.8 # Builder l'image docker
build -t puls-events . # Lancer le container docker run -p 8000:8000 --env-file
.env puls-events
```

Caractéristiques de l'image

Paramètre	Valeur
Image de base	python:3.11-slim
Port exposé	8000
Utilisateur	appuser (non-root)
Variables d'env	Injectées via --env-file .env
Données	Index FAISS pré-construit inclus

11. Pistes d'amélioration

Priorité	Amélioration	Impact
Haute	Enrichissement corpus (+ agendas, + villes)	Score évaluation
Haute	Chunking avancé (découpage fins des descriptions)	Précision RAG
Haute	Filtres par date/lieu/type dans l'API	UX utilisateur
Moyenne	Index FAISS optimisé (HNSW ou IVFFlat)	Performance
Moyenne	Évaluation RAGAS (faithfulness, relevancy)	Métriques QA
Moyenne	CI/CD GitHub Actions (tests + évaluation auto)	Qualité code
Basse	Historique conversationnel (LangChain Memory)	Expérience chat
Basse	Interface Streamlit pour les équipes métier	Adoption
Basse	Authentification JWT sur /rebuild	Sécurité
Basse	Mise à jour automatique de l'index (cron)	Fraîcheur données

12. Conclusion

Ce POC démontre avec succès la faisabilité technique d'un assistant intelligent pour la recommandation d'événements culturels parisiens basé sur une architecture RAG.

Le système atteint un score moyen de similarité de 0.8995 sur un jeu de 10 questions annotées, sans aucune réponse incorrecte. La couverture de tests unitaires est de 100% sur l'ensemble du code, garantissant la robustesse et la maintenabilité de la solution.

L'API REST est fonctionnelle, documentée via Swagger, et entièrement conteneurisée dans une image Docker prête au déploiement. Le code est versionné sur GitHub avec une organisation claire en branches et tags sémantiques.

Les principaux axes d'amélioration identifiés sont l'enrichissement du corpus de données, l'optimisation du chunking, et l'intégration de filtres avancés dans l'API. Ces améliorations permettraient de passer d'un POC à une solution production-ready.